

Build Less, Pull Less, Restart Never: Fast Container Image Distribution and In-Place Updates

Navidreza Asadi
Technical University of Munich
Munich, Germany

Bohdan Garchu
Technical University of Munich
Munich, Germany

Giovanni Bartolomeo
Technical University of Munich
Munich, Germany

Jörg Ott
Technical University of Munich
Munich, Germany

Wolfgang Kellerer
Technical University of Munich
Munich, Germany

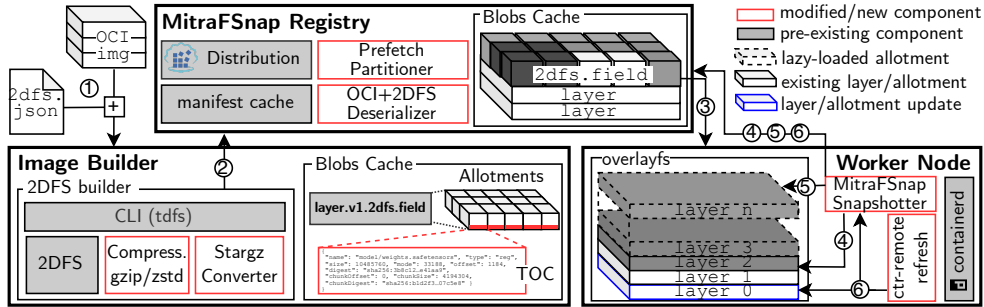


Figure 1. MITRAFSNAP Architecture: the builder creates independently addressable, lazy-loadable allotments; the registry serves a client-selected partition as OCI layers; and the worker prefetches selected content, streams remaining files on demand, and refreshes mounted layers in place.

Motivation

Containers are the de facto standard for packaging and deploying cloud-native applications. Yet the layered OCI image format was conceived for code and dependencies, and it fits poorly with workloads that combine *large artifacts* with *frequent updates*, like machine-learning (ML) serving applications [5]. Application logic changes, model weights are retrained or replaced while the serving process and its dependencies might stay unchanged. When code and dependencies are combined with other artifacts, container build time increases exponentially, updates require rebuilding and redistributing the entire image and cause a disruptive restarts. Additionally, layer-wise updates are difficult to cache and distribute efficiently. The same mismatch appears beyond ML, in cloud-native 5G core components, game engines, and web applications that ship large static assets updated independently of the application. A common workaround decouples the image from its large artifacts and distributes the data separately via external storage or MLOps pipelines, but this adds operational complexity in multi-cloud and edge settings and breaks the reproducibility of a self-contained, versioned image. Existing in-image solutions each solve only part of the problem [1–4]. *2DFS* [1] stores image components as independent, selectable *allotments* on a two-dimensional field, enabling selective builds and independent caching. Unfortunately, selected allotments are still pulled eagerly, and

applying a new version still requires replacing the container. Lazy loading systems such as *eStargz* [2] make files within a layer seekable so the runtime fetches only the chunks it reads, but they optimize only the pull phase over conventional layers, with no independent build or partitioning semantics, and they increase the build time manifold because they require a separate post-build re-compression step. What is missing is a single system that offers reduced overhead at *build time*, efficient *file distribution* and transparent *runtime update* without application restarts while keeping the data inside a reproducible and portable OCI-compatible image (fig. 2).

MITRAFSNAP

We present MITRAFSNAP, an OCI-compatible build, registry, and runtime system for large artifacts distribution in containerized environments (fig. 1). MITRAFSNAP extends the

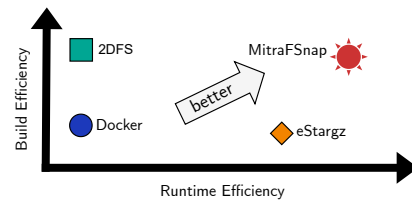


Figure 2. MITRAFSNAP combines the best of two worlds: (1) efficient build and store and (2) efficient runtime.

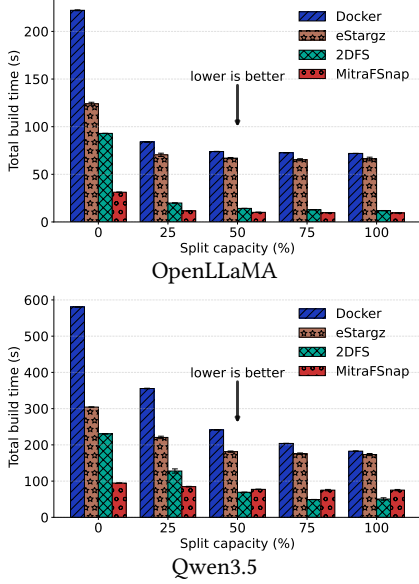


Figure 3. Build performance as available split capacity varies.

containers ecosystem with a unique solution offering a highly efficient build for large images, on-demand file fetching, and live image updates without container restart. We propose a novel architecture built from a combination of 2DFS [1] for the build and storage path and on the stargz-snapshotter [2] for the lazy loading runtime. Additionally, we add a brand new capability for in-place container live updates.

Figure 1 shows the architecture of MITRAFSNAP. This solution spans across three entities: the *Image Builder* that produces images, the *MITRAFSNAP Registry* that stores and serves them, and one or more *Worker Nodes* that pull and run containers. MITRAFSNAP builds on the two-dimensional filesystem of 2DFS, in which each image carries a `2dfs.field` layer indexing its content as allotments on a `<row, col>` grid. An allotment is a self-contained, independently addressable piece of the filesystem. The grid is a coordinate space the developer maps to the application (e.g., model shards along one axis, or weights, drivers, and libraries in separate regions). The builder packages files as independent, lazy loadable allotments, extending the 2DFS layer format with stargz prefetch annotations, preserving fast parallel builds and fine-grained caching. The worker nodes eagerly prefetch an image partition to minimize startup times and lazily access the rest on demand, if needed. Additionally, the worker supports *in-place live refresh* that updates a running container’s mounted data at chunk granularity exploiting a combination of stargz file level TOC (Table of Contents) and 2DFS allotment level metadata. The registry serves the selected image partition as standard OCI layers with prefetch annotations, so any OCI-compliant runtime can pull and run the image, while MITRAFSNAP-aware runtimes can benefit from the lazy loading and live refresh capabilities.

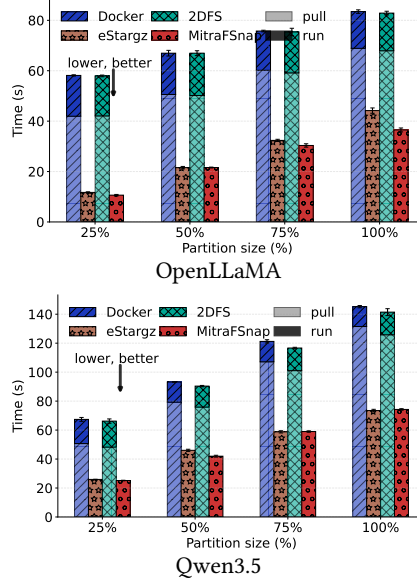


Figure 4. Pull performance as the fraction of model files accessed varies.

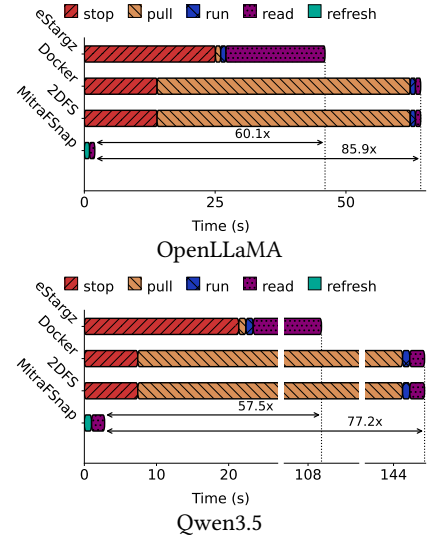


Figure 5. Refresh time compared with replacing the running container.

Evaluation Highlights

We evaluate MITRAFSNAP against *Docker* (eager OCI), *eStargz* (lazy OCI), and *2DFS* (independent allotments; eager) on three language models (OpenLLaMA 3B, Qwen3.5 9B, Gemma-4 31B) and a multi-model video-analytics bundle, in both cloud and resource-constrained edge settings. In this extended abstract, we highlight the build, pull, and live refresh results on a subset of the models.

Build (fig. 3). The Docker and eStargz pipelines are most expensive at coarse packing; partitioning exposes parallel work and reduces their cost. MITRAFSNAP is already fast at low split capacities because allotments build independently. On Qwen3.5 9B, its build time is roughly 75–95 s across capacities, versus 180–580 s for Docker and 175–305 s for eStargz, while staying close to or faster than 2DFS.

Pull (fig. 4). MITRAFSNAP retains the cold-start speed up benefit of lazy access. When Qwen reads 25% of the model, it completes pull plus first access in about 25 s, versus roughly 67 s for Docker and 66 s for 2DFS, and remains close to eStargz throughout the experiment.

Live refresh (fig. 5). This is MITRAFSNAP’s main advantage. All the baselines experience significant delays dominated by terminating the running container, pulling the replacement, and starting it again; MITRAFSNAP instead refreshes the mounted snapshot in place. Across the two models shown, this reduces end-to-end update latency by 57.5–62.5× relative to the manual eStargz path and 69.2–85.9× relative to the full OCI path. Additionally, the refresh time of MITRAFSNAP is not causing downtime but just a brief slowdown for the I/O operations, as the container is still running and serving requests while the update is lazily applied in the background.

References

- [1] Giovanni Bartolomeo, Navidreza Asadi, Wolfgang Kellerer, Jorg Ott, and Nitinder Mohan. 2025. {On-Demand} Container Partitioning for Distributed {ML}. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 1481–1500.
- [2] containerd Authors. [n. d.]. Stargz Snapshotter: Fast Container Image Distribution Plugin with Lazy Pulling (eStargz). <https://github.com/containerd/stargz-snapshotter>. Accessed 2026-06-08.
- [3] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 135–153.
- [4] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, et al. 2025. {ByteCheckpoint}: A unified checkpointing system for large foundation model development. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 559–578.
- [5] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. 2026. {ServeGen}: Workload Characterization and Generation of Large Language Model Serving in Production. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 1845–1859.